

Linear regression

Instructor: Roberto Colomboni

version of 27 September 2026

In the first lecture, we reviewed the mathematical tools that we will use throughout the course. We now turn to our first modelling problem: regression. Here we will introduce what we mean by a predictor, how to measure prediction errors with a loss, and how to fit adjustable parameters to observed data.

The same sequence will recur throughout the course: choose a family of functions, specify a loss, and adjust the parameters to reduce the corresponding empirical loss.

In the sequel, input feature vectors are rows, weights for a single output are columns, and argmin denotes a set of minimisers. The notation $\mathbb{R}^d$ states the number of coordinates, while $\mathbb{R}^{1 \times d}$ and $\mathbb{R}^{d \times 1}$ make the declared orientation explicit when needed.

1 Regression: modelling a relationship and fitting a predictor

Suppose that we have $N \in \mathbb{N}$ observations,

$$(x_1, y_1), \dots, (x_N, y_N),$$

where the index i identifies an observation, x_i is its input and y_i is its observed outcome, also called the *target*. For now, both are real numbers. *Regression* aims to predict a numerical outcome from an input.

A useful starting point is to model each outcome as a systematic relationship plus noise:

$$y_i = f_{\text{true}}(x_i) + \varepsilon_i, \quad i = 1, \dots, N. \quad (1)$$

In (1), f_{true} is an unknown function describing the relationship, and ε_i represents variation around it. Such variation may arise from measurement error or factors not recorded in the input. We observe the pairs $(x_1, y_1), \dots, (x_N, y_N)$, but not the function f_{true} or the individual noise terms.

We might expect the relationship to be approximately described by a simple function. For example, an *affine* function of a scalar input has the form

$$f(x) = ax + b, \quad a, b \in \mathbb{R}.$$

Its graph is a straight line: a is the slope and b the intercept. If the underlying relationship f_{true} is affine, the model becomes

$$y_i = ax_i + b + \varepsilon_i, \quad i = 1, \dots, N. \quad (2)$$

Under (2), our task is to estimate a, b from the observed pairs

$$(x_1, y_1), \dots, (x_N, y_N).$$

We refer to this collection of observed input–target pairs as the *observed data*, or the *dataset*. If we knew the noise terms, we could subtract them from the targets and obtain points on the unknown line. Since we do not know them, we will compare candidate lines by how closely their values match the observed targets.

A *predictor*, also called a *prediction rule*, is a function $f : \mathbb{R} \rightarrow \mathbb{R}$ that assigns a prediction to each input. At the observed input x_i , the prediction is $f(x_i)$. With affine predictors, fitting means choosing the slope and intercept of a candidate line using the observed data. Under the assumed model, the fitted line is our estimate of f_{true} .

More generally, even if we do not know whether f_{true} is affine, we can still choose the family of affine functions and fit one to the observed data: the fitted function then serves as an *approximation* to the unknown relationship.

A small example. Consider these five observations:

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-1.5	0.5	1	2.5	7.5

We use this dataset throughout the scalar-input worked examples.

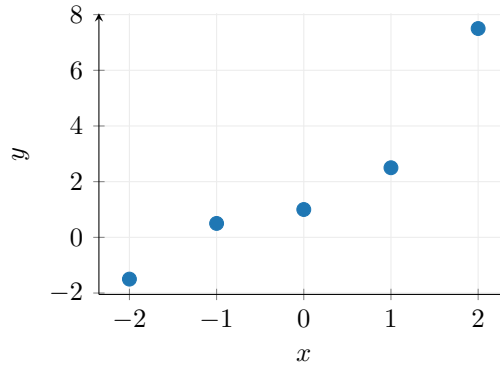


Figure 1: Five observations: each point records an input x_i and its target y_i . We seek a function whose predictions are close to these targets.

The five points are not collinear, so no straight line can pass through all of them. We can nevertheless compare different lines as approximations. As a first candidate, consider

$$f(x) = 1 + 2x. \quad (3)$$

The predictor in (3) has intercept 1 and slope 2. Its predictions at the five inputs are

$$-3, \quad -1, \quad 1, \quad 3, \quad 5.$$

Only the third prediction matches its target exactly. The other predictions lie above or below their targets, as shown in Figure 2.

How should we compare this candidate with another line? We need to specify how prediction errors are penalised and combine the penalties into a numerical criterion. A *loss function* will make this choice precise and allow us to fit the parameters systematically.

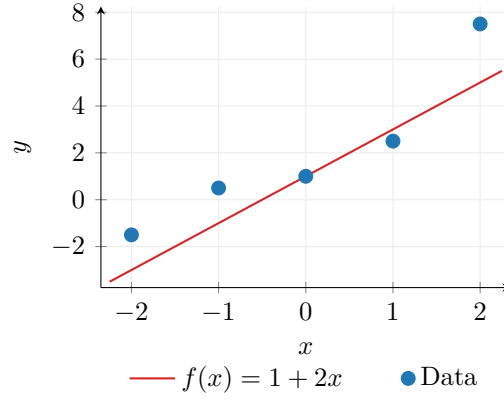


Figure 2: A first guess, $f(x) = 1 + 2x$, plotted against the five data points. Choosing a line gives a prediction at every input, including inputs not present in the dataset.

2 Quantifying prediction errors with a loss

Compare our first guess $f(x) = 1 + 2x$ with $g(x) = 1 - x$. The vertical segments in Figure 3 join each target to its prediction. Their lengths show the prediction errors.

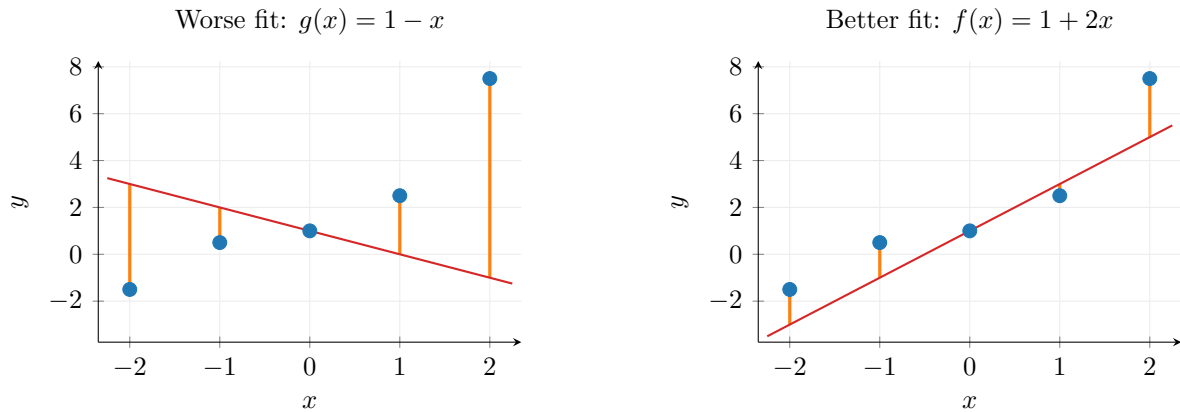


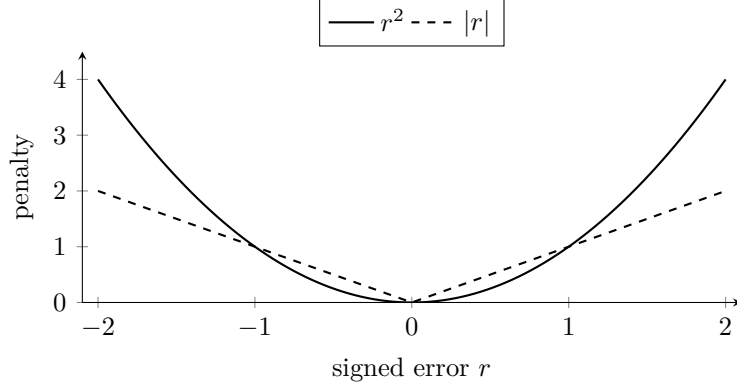
Figure 3: The orange segments show the vertical differences between predictions and targets. Squaring their lengths and adding gives loss 101 for g and loss 11 for f .

A *loss function* assigns a numerical penalty to a prediction and a target. We put the prediction first and the target second. Two choices are the *absolute loss* and the *squared loss*:

$$\ell_{\text{abs}}(\hat{y}, y) := |\hat{y} - y|, \quad \ell_{\text{sq}}(\hat{y}, y) := (\hat{y} - y)^2. \quad (4)$$

Both losses in (4) are nonnegative and equal zero exactly when the prediction equals the target. They differ in how they penalise errors. Doubling the magnitude of an error doubles its absolute loss but quadruples its squared loss.

If we define the *residual* $r = \hat{y} - y$, the two penalties are $|r|$ and r^2 . The first has a corner at zero, while the second is differentiable everywhere, with derivative $2r$ with respect to r . The corner does not make absolute error an invalid objective, but it means that ordinary derivatives alone will not necessarily settle every optimisation question. We return to this in Exercises 4 and 5.



2.1 From individual errors to an objective on parameters

Let the column vector $\boldsymbol{\theta} \in \mathbb{R}^p$ collect the p adjustable parameters of a family of predictors. We write $f_{\boldsymbol{\theta}}$ for the function associated with the parameter choice $\boldsymbol{\theta}$. For fixed observations $(x_1, y_1), \dots, (x_N, y_N)$, the *sum-of-squared-errors objective* is

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}(x_i), y_i) = \sum_{i=1}^N (f_{\boldsymbol{\theta}}(x_i) - y_i)^2. \quad (5)$$

The function $\mathcal{L}_{\text{sq}}$ is the *empirical loss function* associated with the predictor family $\boldsymbol{\theta} \mapsto f_{\boldsymbol{\theta}}$, the loss function ℓ_{sq} , and the dataset $(x_1, y_1), \dots, (x_N, y_N)$. It aggregates the losses over the observed dataset and is the objective that we aim to minimise as a function of the parameter $\boldsymbol{\theta}$.

The loss ℓ_{sq} takes one prediction and one target. The empirical loss $\mathcal{L}_{\text{sq}}$ takes the parameters, uses them to make all the predictions, and adds the resulting losses.

The subscript serves as a reminder of the choice of loss. When we need to distinguish several predictor families, we add a superscript identifying the family to the objective, writing, for example, $\mathcal{L}_{\text{sq}}^{\text{const}}$, $\mathcal{L}_{\text{sq}}^{\text{slope}}$, or $\mathcal{L}_{\text{sq}}^{\text{aff}}$. When the family is clear from the context, we omit the superscript. Choosing the predictor family and choosing the loss are separate decisions.

Throughout an optimisation calculation, the inputs and targets are fixed, while the parameters vary.

We call

$$r_i(\boldsymbol{\theta}) := f_{\boldsymbol{\theta}}(x_i) - y_i \quad (6)$$

the *residual*, using the sign convention “prediction minus target”. Then, notice that it holds that $\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \sum_{i=1}^N r_i(\boldsymbol{\theta})^2$. Positive residuals in (6) indicate overprediction, negative residuals underprediction. A residual is computable once we choose the predictor.

When evaluating our performance, we do not want opposite signed errors to simply cancel: in a regression problem, we consider both overprediction and underprediction as mistakes, and this is one reason for applying a nonnegative penalty (such as the square of the absolute value) before summing.

Worked comparison. For the two predictors compared in Figure 3, the residuals from (6) are

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	sum of squares
$f(x_i) - y_i$	-1.5	-1.5	0	0.5	-2.5	11
$g(x_i) - y_i$	4.5	1.5	0	-2.5	-8.5	101

We observe that squared error agrees with our visual comparison: intuitively, f seems to fit these observations better than g .

2.2 The empirical loss: other conventions and interpretation

In the literature, other common objectives, corresponding to different normalisations, are

$$\frac{1}{N}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}), \quad \frac{1}{2}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}), \quad \frac{1}{2N}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}).$$

For a fixed dataset, all have exactly the same minimisers. Indeed, multiplying an objective by a positive constant preserves every comparison between its values. Their *gradients*, however, are multiplied by that constant too.

We remark that *fitting the observed data* and *predicting accurately on unseen observations* are different questions. In particular, a small value of (5) is a statement about the observations in that sum, i.e., the already seen data, and is not by itself a guarantee of accurate predictions on unseen data.

The ability of a fitted predictor to perform well on unseen data is called *generalisation*. Phenomena such as *underfitting* and *overfitting* can prevent a model from generalising well. In this lecture, however, we will focus only on understanding the fitting calculation itself.

3 Choosing a family of prediction functions

The goal of this section is to use derivatives/gradients to compute the best predictor in the chosen class that fits the observed data according to the chosen loss. We have already considered straight-line predictors, but to learn the ropes we compare that family with two simpler families, each imposing a restriction on the coefficients:

family	prediction at x	parameters
constant	b	$b \in \mathbb{R}$
line through the origin	ax	$a \in \mathbb{R}$
general straight line	$b + ax$	$(b, a) \in \mathbb{R}^2$

The slope a controls the change in the prediction when x increases by one. The intercept b is the prediction at $x = 0$. The term *bias* is also used for an intercept. Technically, a straight line with a nonzero intercept is an *affine* function of x , rather than a linear map in the strict linear-algebra sense.

Choosing the family of predictors and choosing its parameters are separate decisions. A constant predictor cannot change with x , no matter how well we choose b , while a line through the origin is forced to predict zero at $x = 0$. A general straight line removes both of these restrictions, but it still cannot bend.

Each of the families above contains infinitely many functions. We seek parameter values that minimise the empirical loss. As in Lecture 1, we write

$$\boldsymbol{\theta}^* \in \underset{\boldsymbol{\theta} \in \mathbb{R}^p}{\operatorname{argmin}} \mathcal{L}_{\text{sq}}(\boldsymbol{\theta})$$

for a minimiser.

We recall that, by Fermat's theorem, the gradient of a differentiable objective is zero at a local minimum (i.e., the point is stationary), and that this condition is necessary but not sufficient:

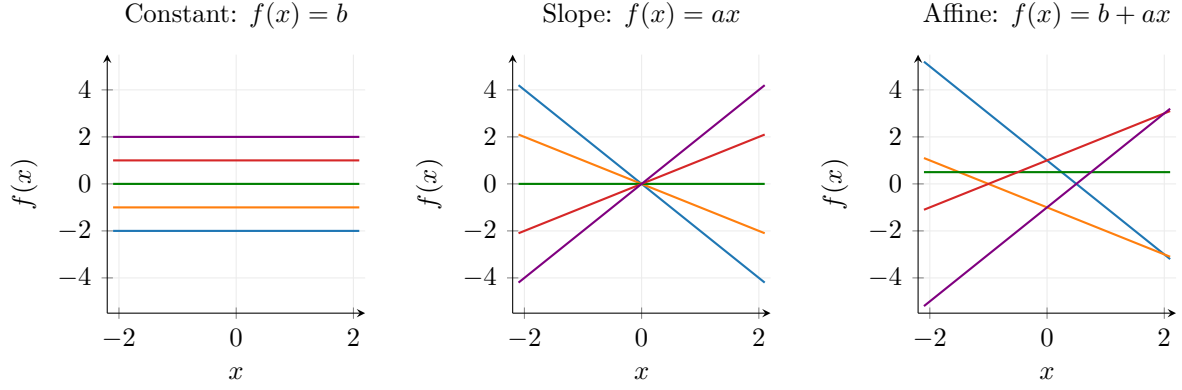


Figure 4: Different parameter choices give different functions within each family. Changing b moves a line vertically, while changing a changes its slope. The affine family allows both to vary.

a stationary point need not be a minimum. So, our strategy will be to first set the gradient to zero to find the stationary points, and then understand whether or not the points we have found this way are in fact the minima we were seeking.

3.1 The best constant predictor

The constant predictor $f_b^{\text{const}} : \mathbb{R} \rightarrow \mathbb{R}$ ignores its input:

$$f_b^{\text{const}}(x) := b. \quad (7)$$

Using (5), its squared-error objective is the one-variable function

$$\mathcal{L}_{\text{sq}}^{\text{const}}(b) := \sum_{i=1}^N \ell_{\text{sq}}(f_b^{\text{const}}(x_i), y_i) = \sum_{i=1}^N (b - y_i)^2. \quad (8)$$

Figure 5 connects a horizontal prediction line to a point on the graph of the objective (8). Changing b moves the prediction line vertically, while on the right, b is the (horizontal) coordinate of the empirical loss function.

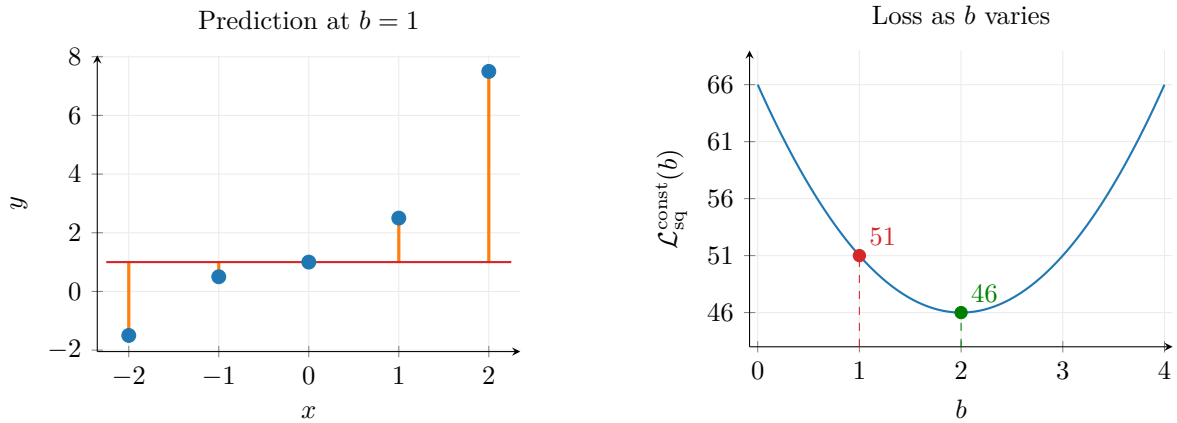


Figure 5: Left: the input axis is x , and the constant prediction $b = 1$ gives squared loss 51. Right: the horizontal axis is the parameter b , and the curve is $\mathcal{L}_{\text{sq}}^{\text{const}}(b) = 5b^2 - 20b + 66$. Its minimum is 46, attained at $b = 2$.

To differentiate (8), define

$$r_i(b) = b - y_i, \quad q(s) = s^2. \quad (9)$$

In (9), $r'_i(b) = 1$, because y_i is fixed, and $q'(s) = 2s$. Applying the chain rule to each term gives

$$(\mathcal{L}_{\text{sq}}^{\text{const}})'(b) = \sum_{i=1}^N q'(r_i(b)) r'_i(b) = 2 \sum_{i=1}^N (b - y_i) = 2 \left(Nb - \sum_{i=1}^N y_i \right). \quad (10)$$

In the final equality of (10), we used the fact that b does not depend on i .

We now set (10) to zero (to find the stationary points) and obtain

$$Nb = \sum_{i=1}^N y_i. \quad (11)$$

Since $N \geq 1$, solving (11) gives the sample mean

$$b^* = \bar{y}, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i. \quad (12)$$

We now prove that the stationary point b^* is in fact the global minimiser. Notice that (8) is a quadratic in b with positive leading coefficient N . Hence, its graph is an upward-opening parabola, so the objective admits a global minimiser. By Fermat's theorem, any global minimiser must be a stationary point, and since b^* is the only stationary point, it follows that b^* is the global minimiser:

$$\operatorname{argmin}_{b \in \mathbb{R}} \mathcal{L}_{\text{sq}}^{\text{const}}(b) = \{b^*\} = \{\bar{y}\}.$$

Worked example. Applying (12) to our five targets gives

$$b^* = \frac{-1.5 + 0.5 + 1 + 2.5 + 7.5}{5} = 2.$$

Its residuals are 3.5, 1.5, 1, -0.5 , -5.5 , giving loss

$$3.5^2 + 1.5^2 + 1^2 + (-0.5)^2 + (-5.5)^2 = 46.$$

We remark that, for a fixed predictor family and dataset, the choice of the loss function can change the value of the best parameter. In particular, *the fact that the best parameter here is the sample mean $\bar{y}$ is a consequence of our choice of the squared error as the loss function.*

We will see in Exercise 5 that choosing the absolute error as our loss function instead leads to a *median* of the targets. Thus, what is considered “best” depends crucially on the choice of the loss function.

3.2 The best line through the origin

Next consider a line through the origin:

$$f_a^{\text{slope}}(x) := ax. \quad (13)$$

Using (5), its squared-error objective is the one-variable function

$$\mathcal{L}_{\text{sq}}^{\text{slope}}(a) := \sum_{i=1}^N \ell_{\text{sq}}(f_a^{\text{slope}}(x_i), y_i) = \sum_{i=1}^N (ax_i - y_i)^2. \quad (14)$$

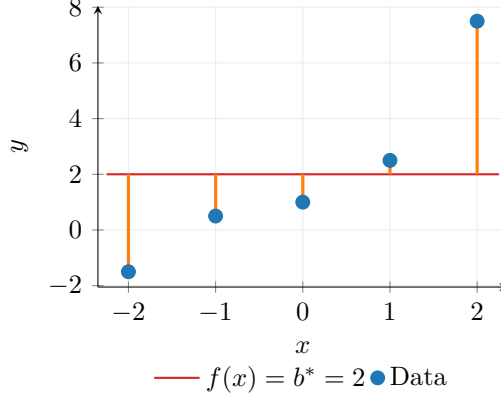


Figure 6: The best constant prediction is the mean of the targets, $b^* = (-1.5 + 0.5 + 1 + 2.5 + 7.5)/5 = 2$. It minimises the sum of squared vertical residuals, giving loss 46.

To differentiate (14), define

$$r_i(a) = ax_i - y_i, \quad q(s) = s^2. \quad (15)$$

In (15), $r'_i(a) = x_i$, because x_i and y_i are fixed, and $q'(s) = 2s$. Applying the chain rule to each term gives

$$(\mathcal{L}_{\text{sq}}^{\text{slope}})'(a) = \sum_{i=1}^N q'(r_i(a)) r'_i(a) = 2 \sum_{i=1}^N (ax_i - y_i) x_i = 2 \left(a \sum_{i=1}^N x_i^2 - \sum_{i=1}^N x_i y_i \right). \quad (16)$$

Assume first that

$$\sum_{i=1}^N x_i^2 > 0.$$

We set (16) to zero (to find the stationary points) and obtain

$$a \sum_{i=1}^N x_i^2 = \sum_{i=1}^N x_i y_i. \quad (17)$$

Solving (17) gives the stationary point

$$a^* = \frac{\sum_{i=1}^N x_i y_i}{\sum_{i=1}^N x_i^2}. \quad (18)$$

We now prove that the stationary point a^* is in fact the global minimiser. Notice that (14) is a quadratic in a with positive leading coefficient $\sum_{i=1}^N x_i^2$. Hence, its graph is an upward-opening parabola, so the objective admits a global minimiser. By Fermat's theorem, any global minimiser must be a stationary point, and since a^* is the only stationary point, it follows that a^* is the global minimiser:

$$\operatorname{argmin}_{a \in \mathbb{R}} \mathcal{L}_{\text{sq}}^{\text{slope}}(a) = \{a^*\} = \left\{ \frac{\sum_{i=1}^N x_i y_i}{\sum_{i=1}^N x_i^2} \right\}.$$

If instead

$$\sum_{i=1}^N x_i^2 = 0,$$

then $x_i = 0$ for every i . Every value of a therefore gives the same predictions, all equal to zero, and the same loss

$$\mathcal{L}_{\text{sq}}^{\text{slope}}(a) = \sum_{i=1}^N y_i^2.$$

In this case, formula (18) is unavailable and every $a \in \mathbb{R}$ is a minimiser.

Worked example. For our five observations,

$$\sum_{i=1}^5 x_i^2 = 10, \quad \sum_{i=1}^5 x_i y_i = 20, \quad a^* = 2.$$

The fitted predictions are $-4, -2, 0, 2, 4$, with residuals $-2.5, -2.5, -1, -0.5, -3.5$, giving loss 26. This is smaller than the best constant loss 46. The two families impose different restrictions, so either can give the smaller fitted loss on a different dataset.

3.3 Fitting an intercept and a slope together

We now consider a general straight line. Collect the intercept and slope in the parameter vector

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ a \end{pmatrix},$$

and define

$$f_{\boldsymbol{\theta}}^{\text{aff}}(x) := b + ax. \quad (19)$$

Using (5), its squared-error objective is

$$\mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}) := \mathcal{L}_{\text{sq}}^{\text{aff}}(b, a) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}^{\text{aff}}(x_i), y_i) = \sum_{i=1}^N (b + ax_i - y_i)^2. \quad (20)$$

To differentiate (20), define

$$r_i(b, a) = b + ax_i - y_i, \quad q(s) = s^2. \quad (21)$$

Since the data are fixed,

$$\frac{\partial r_i}{\partial b} = 1, \quad \frac{\partial r_i}{\partial a} = x_i, \quad q'(s) = 2s.$$

Applying the chain rule to each term gives

$$\frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial b}(b, a) = 2 \sum_{i=1}^N (b + ax_i - y_i) = 2 \left(Nb + a \sum_{i=1}^N x_i - \sum_{i=1}^N y_i \right), \quad (22)$$

$$\frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial a}(b, a) = 2 \sum_{i=1}^N x_i (b + ax_i - y_i) = 2 \left(b \sum_{i=1}^N x_i + a \sum_{i=1}^N x_i^2 - \sum_{i=1}^N x_i y_i \right). \quad (23)$$

We now set the gradient to zero (to find the stationary points). Since the gradient has two coordinates, this means setting both partial derivatives in (22)–(23) to zero. Dividing by 2 gives the stationarity conditions

$$\begin{aligned} Nb + a \sum_{i=1}^N x_i &= \sum_{i=1}^N y_i, \\ b \sum_{i=1}^N x_i + a \sum_{i=1}^N x_i^2 &= \sum_{i=1}^N x_i y_i. \end{aligned} \quad (24)$$

Write the sample means as

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i.$$

The first equation in (24) gives

$$b = \bar{y} - a\bar{x}. \quad (25)$$

Substituting (25) into the second stationarity condition gives

$$a \left(\sum_{i=1}^N x_i^2 - N\bar{x}^2 \right) = \sum_{i=1}^N x_i y_i - N\bar{x}\bar{y}. \quad (26)$$

Assume first that the inputs are not all equal. Then

$$\sum_{i=1}^N x_i^2 - N\bar{x}^2 = \sum_{i=1}^N (x_i - \bar{x})^2 > 0,$$

so (26) has a unique solution. Hence, the unique stationary point is

$$\boxed{\begin{aligned} a^* &= \frac{\sum_{i=1}^N x_i y_i - N\bar{x}\bar{y}}{\sum_{i=1}^N x_i^2 - N\bar{x}^2} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}, \\ b^* &= \bar{y} - a^*\bar{x}. \end{aligned}} \quad (27)$$

Equivalently, the fitted line can be written as

$$f_{\boldsymbol{\theta}^*}^{\text{aff}}(x) = \bar{y} + a^*(x - \bar{x}), \quad \boldsymbol{\theta}^* = \begin{pmatrix} b^* \\ a^* \end{pmatrix}. \quad (28)$$

In particular, notice that the fitted line passes through $(\bar{x}, \bar{y})$.

We now prove that the stationary point $\boldsymbol{\theta}^*$ is in fact the global minimiser. Its residuals are

$$r_i^* = b^* + a^*x_i - y_i, \quad i = 1, \dots, N.$$

With this notation, the two zero-gradient conditions in (22)–(23) say precisely that

$$\sum_{i=1}^N r_i^* = 0, \quad \sum_{i=1}^N x_i r_i^* = 0. \quad (29)$$

Now, any other parameter pair can be written as $(b^* + u, a^* + v)$, where $u, v \in \mathbb{R}$. Its residual at observation i is then

$$(b^* + u) + (a^* + v)x_i - y_i = r_i^* + u + vx_i. \quad (30)$$

Expanding the squares and using (29) gives

$$\begin{aligned} \mathcal{L}_{\text{sq}}^{\text{aff}}(b^* + u, a^* + v) &= \sum_{i=1}^N (r_i^* + u + vx_i)^2 \\ &= \mathcal{L}_{\text{sq}}^{\text{aff}}(b^*, a^*) + \underbrace{2u \sum_{i=1}^N r_i^*}_{=0} + \underbrace{2v \sum_{i=1}^N x_i r_i^*}_{=0} + \underbrace{\sum_{i=1}^N (u + vx_i)^2}_{\geq 0} \geq \mathcal{L}_{\text{sq}}^{\text{aff}}(b^*, a^*). \end{aligned}$$

Thus θ^* is a global minimiser. Since the inputs are not all equal, it is the unique stationary point and hence the unique global minimiser.

If instead all inputs are equal to some value c , every prediction is $b + ac$, so the objective depends on b and a only through this quantity. From Section 3.1, the best constant prediction under squared error is $\bar{y}$. Hence,

$$\operatorname{argmin}_{(b,a) \in \mathbb{R}^2} \mathcal{L}_{\text{sq}}^{\text{aff}}(b,a) = \{(b,a) \in \mathbb{R}^2 : b + ac = \bar{y}\}.$$

Hence, in this latter case, the fitted predictions are unique, but the slope and intercept are not.

Worked example. For our five observations, $\bar{x} = 0$ and $\bar{y} = 2$, so

$$a^* = \frac{\sum_{i=1}^5 x_i y_i}{\sum_{i=1}^5 x_i^2} = \frac{20}{10} = 2, \quad b^* = 2 - 2 \cdot 0 = 2.$$

Thus the fitted line is

$$f_{\theta^*}^{\text{aff}}(x) = 2 + 2x.$$

Stationary conditions in matrix notation. To become more familiar with matrix notation, notice that the stationarity conditions (24) can also be written in matrix form:

$$\begin{pmatrix} N & \sum_{i=1}^N x_i \\ \sum_{i=1}^N x_i & \sum_{i=1}^N x_i^2 \end{pmatrix} \begin{pmatrix} b \\ a \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^N y_i \\ \sum_{i=1}^N x_i y_i \end{pmatrix}. \quad (31)$$

The next section generalises (31) to several input features.

4 Regression with several input features

So far, each observation has had a single input x , and our most flexible predictor has been the straight line $b + ax$. However, an observation may come with several measurements that could help us predict its target. We can use all of them by giving each input feature its own weight, while keeping the intercept.

Write the input as a row $\mathbf{x} = (x_1, \dots, x_d)$, where $d \geq 1$ is the number of input features, and let $\mathbf{w} = (w_1, \dots, w_d)^\top$ be the column of their weights. We collect all the coefficients that we want to fit in one column, putting the intercept first:

$$\boldsymbol{\theta} := \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix} = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix} \in \mathbb{R}^{(d+1) \times 1}. \quad (32)$$

Thus there are $d + 1$ coefficients to choose: the intercept and one weight for each input feature.

The predictor is then defined as

$$f_{\boldsymbol{\theta}}(\mathbf{x}) := b + \mathbf{x}\mathbf{w} = b + w_1 x_1 + \dots + w_d x_d. \quad (33)$$

The target we aim to predict is still a real number. When $d = 1$, writing $w_1 = a$ gives exactly the straight-line predictor from Section 3.3. With two input features, on the other hand, the predictor is $b + w_1 x_1 + w_2 x_2$, so each of the two measurements can contribute to the prediction.

4.1 One matrix for all the predictions

For observations $(\mathbf{x}_i, y_i)$, $i = 1, \dots, N$, define the *design matrix* and the column of targets by

$$\mathbf{X} := \begin{pmatrix} 1 & \mathbf{x}_1 \\ \vdots & \vdots \\ 1 & \mathbf{x}_N \end{pmatrix} \in \mathbb{R}^{N \times (d+1)}, \quad \mathbf{y} := \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{N \times 1}. \quad (34)$$

Each row of $\mathbf{X}$ corresponds to one observation. Its first entry is 1, and the remaining entries are that observation's input features. The first column therefore multiplies the intercept, while the other columns multiply the feature weights. In particular, the row for observation i gives

$$\begin{pmatrix} 1 & \mathbf{x}_i \end{pmatrix} \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix} = b + \mathbf{x}_i \mathbf{w} = f_{\boldsymbol{\theta}}(\mathbf{x}_i).$$

Stacking these row calculations gives the column vector with all the predictions at once:

$$\mathbf{X}\boldsymbol{\theta} = \begin{pmatrix} b + \mathbf{x}_1 \mathbf{w} \\ \vdots \\ b + \mathbf{x}_N \mathbf{w} \end{pmatrix} = \begin{pmatrix} f_{\boldsymbol{\theta}}(\mathbf{x}_1) \\ \vdots \\ f_{\boldsymbol{\theta}}(\mathbf{x}_N) \end{pmatrix}. \quad (35)$$

As a sanity check, we see that the dimensions are in fact compatible:

$$\underbrace{\mathbf{X}}_{N \times (d+1)} \underbrace{\boldsymbol{\theta}}_{(d+1) \times 1} \text{ has shape } N \times 1.$$

There is one predicted value for each observation, just as there is one target in each entry of $\mathbf{y}$.

We can now subtract the targets to obtain the residual column:

$$\mathbf{r}(\boldsymbol{\theta}) := \mathbf{X}\boldsymbol{\theta} - \mathbf{y}. \quad (36)$$

Its i -th entry is the prediction minus the target for observation i , and hence it is the i -th residual. Squaring these entries and adding them gives the same empirical loss as before, now written in matrix form:

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \sum_{i=1}^N r_i(\boldsymbol{\theta})^2 = \|\mathbf{r}(\boldsymbol{\theta})\|_2^2 = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2. \quad (37)$$

4.2 The least-squares formula and its dimensions

Minimising (as a function of the parameter $\boldsymbol{\theta}$) the sum of squared residuals is called *least-squares fitting*. We will follow the same steps as for the straight line: first compute the gradient, then set it to zero (to find the stationary points) and check that the resulting stationary points are global minimisers. The matrix notation lets us do this for all the coefficients together, including the intercept. Later, in Section 5.3, we will use the same gradient for *gradient descent*.

Differentiating a residual. To find one entry of the gradient, vary one coefficient θ_j and hold the other coefficients fixed. Here $\theta_1 = b$, while $\theta_{j+1} = w_j$ for $j = 1, \dots, d$. In terms of the design matrix, the residual at observation i is

$$r_i(\boldsymbol{\theta}) = \sum_{k=1}^{d+1} X_{ik} \theta_k - y_i.$$

Since

$$\frac{\partial \theta_k}{\partial \theta_j} = \delta_{kj},$$

where δ_{kj} is the Kronecker delta, by the linearity of the derivative, we obtain

$$\frac{\partial r_i}{\partial \theta_j}(\boldsymbol{\theta}) = \frac{\partial}{\partial \theta_j} \left(\sum_{k=1}^{d+1} X_{ik} \theta_k - y_i \right) = \sum_{k=1}^{d+1} X_{ik} \frac{\partial \theta_k}{\partial \theta_j} = \sum_{k=1}^{d+1} X_{ik} \delta_{kj} = X_{ij}, \quad j = 1, \dots, d+1. \quad (38)$$

In particular, the derivative with respect to the intercept b (i.e., θ_1) is 1, because the first column of $\mathbf{X}$ consists of ones. This is the same calculation that we used for the intercept of a straight line.

Differentiating the objective. Each observation contributes the square of its residual. Applying the chain rule to this square and then adding over the observations gives

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial \theta_j}(\boldsymbol{\theta}) = \frac{\partial}{\partial \theta_j} \sum_{i=1}^N (r_i(\boldsymbol{\theta}))^2 = \sum_{i=1}^N \frac{\partial}{\partial \theta_j} (r_i(\boldsymbol{\theta}))^2 = \sum_{i=1}^N 2r_i(\boldsymbol{\theta}) \frac{\partial r_i}{\partial \theta_j}(\boldsymbol{\theta}) = 2 \sum_{i=1}^N X_{ij} r_i(\boldsymbol{\theta}). \quad (39)$$

Thus the derivative for coefficient j uses column j of the design matrix: multiply each residual by the corresponding entry of that column, and add the results. This sum is also entry j of a matrix–vector product. Indeed,

$$(\mathbf{X}^\top \mathbf{r}(\boldsymbol{\theta}))_j = \sum_{i=1}^N (\mathbf{X}^\top)_{ji} r_i(\boldsymbol{\theta}) = \sum_{i=1}^N X_{ij} r_i(\boldsymbol{\theta}) = \frac{1}{2} \frac{\partial \mathcal{L}_{\text{sq}}}{\partial \theta_j}(\boldsymbol{\theta}). \quad (40)$$

Consequently, collecting the partial derivatives in their coefficient order gives

$$\nabla \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial \mathcal{L}_{\text{sq}}}{\partial \theta_1}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial \mathcal{L}_{\text{sq}}}{\partial \theta_{d+1}}(\boldsymbol{\theta}) \end{pmatrix} = 2 \begin{pmatrix} (\mathbf{X}^\top \mathbf{r}(\boldsymbol{\theta}))_1 \\ \vdots \\ (\mathbf{X}^\top \mathbf{r}(\boldsymbol{\theta}))_{d+1} \end{pmatrix} = 2 \mathbf{X}^\top \mathbf{r}(\boldsymbol{\theta}) = 2 \mathbf{X}^\top (\mathbf{X} \boldsymbol{\theta} - \mathbf{y}). \quad (41)$$

As a sanity check, notice that the shape of the matrix $\mathbf{X}^\top$ is $(d+1) \times N$ and that of the column vector $\mathbf{X} \boldsymbol{\theta} - \mathbf{y}$ is $N \times 1$ and, as expected, the gradient has one entry for each coefficient, with the intercept derivative first.

Finding the stationary points. A stationary point must make every entry of the gradient zero. By (41), this means

$$\mathbf{X}^\top (\mathbf{X} \boldsymbol{\theta} - \mathbf{y}) = \mathbf{0}.$$

Rearranging gives the linear system known as the *normal equations*:

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}. \quad (42)$$

These are $d+1$ equations in the $d+1$ coefficients (the unknown vector is $\boldsymbol{\theta} \in \mathbb{R}^{(d+1) \times 1}$). Their solutions are exactly the stationary points of the empirical squared loss. We still need to check that they really minimise it.

Why is a stationary point a global minimiser? Suppose that $\boldsymbol{\theta}^*$ solves (42), and write $\mathbf{r}^* = \mathbf{X}\boldsymbol{\theta}^* - \mathbf{y}$ for its residual column. With this definition, notice that the normal equations can be read as saying that $\mathbf{X}^\top \mathbf{r}^* = \mathbf{0}$. As in the straight-line calculation, we compare this candidate with any other coefficient choice. Such a choice can be written as $\boldsymbol{\theta}^* + \mathbf{v}$, where $\mathbf{v} \in \mathbb{R}^{(d+1) \times 1}$ contains the changes to the coefficients.

The new residual column is $\mathbf{r}^* + \mathbf{X}\mathbf{v}$. Expanding its squared norm yields

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}^* + \mathbf{v}) = \|\mathbf{r}^* + \mathbf{X}\mathbf{v}\|_2^2 = \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}^*) + 2\mathbf{v}^\top \underbrace{\mathbf{X}^\top \mathbf{r}^*}_{=\mathbf{0}} + \underbrace{\|\mathbf{X}\mathbf{v}\|_2^2}_{\geq 0} \geq \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}^*). \quad (43)$$

Since this holds for every change $\mathbf{v}$, no other coefficient choice has a smaller loss. Thus every stationary point of this least-squares objective is a global minimiser.

The explicit formula. If the matrix $\mathbf{X}^\top \mathbf{X}$ is invertible,¹ we can use the same strategy we saw in Lecture 1 to solve the linear system given by the normal equations. Specifically, we can multiply on the left by $(\mathbf{X}^\top \mathbf{X})^{-1}$ to obtain

$$\boxed{\boldsymbol{\theta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.} \quad (44)$$

Since $\mathbf{X}^\top \mathbf{X}$ is invertible, the normal equations have a unique solution. As we showed above, every stationary point of the least-squares objective is a global minimiser. Therefore, $\boldsymbol{\theta}^*$ is the unique global minimiser. Its first entry is the fitted intercept, while its remaining entries are the fitted feature weights.

As another sanity check, it is worth checking the dimensions of the two quantities in the normal equations:

$$\underbrace{\mathbf{X}^\top \mathbf{X}}_{(d+1) \times (d+1)}, \quad \underbrace{\mathbf{X}^\top \mathbf{y}}_{(d+1) \times 1}.$$

The inverse in (44) therefore multiplies a column of length $d + 1$ and returns a column of the same length, as required for $\boldsymbol{\theta}^*$. On the other hand, $\mathbf{X}\mathbf{X}^\top$ has shape $N \times N$ and it is *not* the matrix in the normal equations: the two products need not coincide, even when they happen to have the same dimensions.

If $\mathbf{X}^\top \mathbf{X}$ is not invertible, the inverse formula in (44) is unavailable. The normal equations still have solutions, but the fitted coefficients are no longer unique. Nevertheless, all their solutions give the same fitted predictions on the observed inputs. This is the same distinction we encountered when all scalar inputs were equal in Section 3.3.

4.3 A one-feature example: fitting a line

To check that this construction really extends what we already know, return to the five scalar-input observations from our worked examples. There is now just one input feature, so the design matrix has two columns, one for the intercept and one for the slope:

$$\mathbf{X} = \begin{pmatrix} 1 & -2 \\ 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad \boldsymbol{\theta} = \begin{pmatrix} b \\ a \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} -1.5 \\ 0.5 \\ 1 \\ 2.5 \\ 7.5 \end{pmatrix}. \quad (45)$$

¹As an optional comment for the mathematically inclined reader: an equivalent condition is that the columns of $\mathbf{X}$ are *linearly independent*, i.e., that the only way to take a linear combination of the columns of $\mathbf{X}$ and obtain the zero vector is to multiply every column by zero. See Appendix A.

Multiplying a row of $\mathbf{X}$ by $\boldsymbol{\theta}$ gives $b + ax_i$, exactly as before. The normal equations (42) are therefore the matrix system (31) from Section 3.3. For these observations, the two products are

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 5 & 0 \\ 0 & 10 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 10 \\ 20 \end{pmatrix}.$$

Using (44) gives

$$\boldsymbol{\theta}^* = \begin{pmatrix} b^* \\ a^* \end{pmatrix} = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/10 \end{pmatrix} \begin{pmatrix} 10 \\ 20 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

Thus we recover the fitted line $f_{\boldsymbol{\theta}^*}^{\text{aff}}(x) = 2 + 2x$. Its predictions are $-2, 0, 2, 4, 6$, and its loss is

$$(-0.5)^2 + (-0.5)^2 + 1^2 + 1.5^2 + (-1.5)^2 = 6.$$

A model without an intercept. If we choose a predictor without an intercept, such as the line through the origin from Section 3.2, we simply leave out the column of ones and its coefficient. The same matrix calculation still applies to the remaining columns and coefficients. In fact, the derivative and minimum arguments above only used the fact that predictions are a fixed matrix multiplied by a coefficient column. This observation will also let us use features constructed from the original inputs.

4.4 A two-feature example: fitting a plane

With two input features, the graph of the fitted predictor is a plane rather than a line. As an example, consider the following nine observations:

x_1	x_2	y
-1	-1	-2
-1	0	-1
-1	1	0
0	-1	-2
0	0	0
0	1	2
1	-1	-2
1	0	1
1	1	4

Since we include an intercept, the corresponding design matrix and output vector are

$$\mathbf{X} = \begin{pmatrix} 1 & -1 & -1 \\ 1 & -1 & 0 \\ 1 & -1 & 1 \\ 1 & 0 & -1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & -1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} -2 \\ -1 \\ 0 \\ -2 \\ 0 \\ 2 \\ -2 \\ 1 \\ 4 \end{pmatrix}.$$

Now, computations show that

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 9 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 6 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 0 \\ 6 \\ 12 \end{pmatrix}.$$

Since $\mathbf{X}^\top \mathbf{X}$ is diagonal, its inverse is particularly simple:

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \begin{pmatrix} 1/9 & 0 & 0 \\ 0 & 1/6 & 0 \\ 0 & 0 & 1/6 \end{pmatrix}.$$

Therefore,

$$\boldsymbol{\theta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 1/9 & 0 & 0 \\ 0 & 1/6 & 0 \\ 0 & 0 & 1/6 \end{pmatrix} \begin{pmatrix} 0 \\ 6 \\ 12 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 2 \end{pmatrix}.$$

Recalling that

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ w_1 \\ w_2 \end{pmatrix},$$

we obtain

$$b^* = 0, \quad w_1^* = 1, \quad w_2^* = 2.$$

Hence the fitted predictor is

$$f_{\boldsymbol{\theta}^*}(x_1, x_2) = x_1 + 2x_2.$$

Figure 7 shows the resulting fitted plane and the residuals of the nine observations.

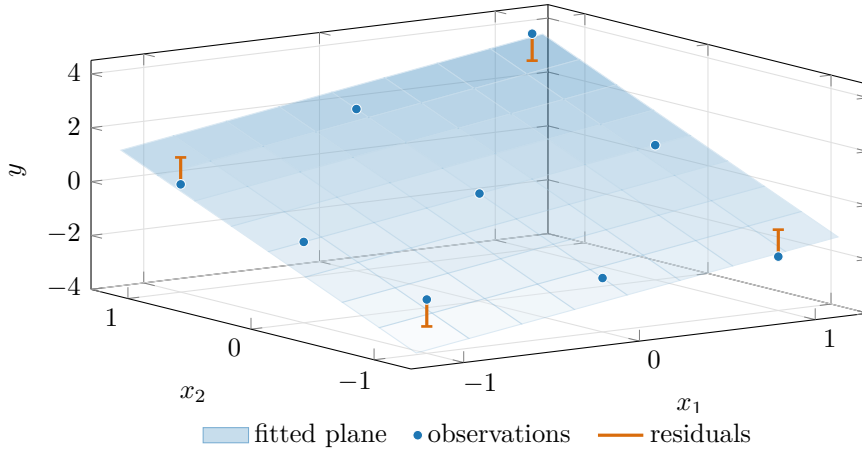


Figure 7: With two input features, an affine predictor describes a plane. For these nine observations, the least-squares fit is $f_{\boldsymbol{\theta}^*}(x_1, x_2) = x_1 + 2x_2$, with $\boldsymbol{\theta}^* = (0, 1, 2)^\top$. The first entry is the fitted intercept, which is zero in this example. The orange segments join each observation to its prediction on the plane, keeping both input coordinates fixed. Their lengths are the absolute residuals. Five observations lie exactly on the plane and have zero residual.

4.5 Regression with nonlinear features

So far, when working with a single input x , our predictors have been affine functions of x , corresponding to fitting a straight line. However, a straight line may be too restrictive when the relationship between the input and the output is curved. For example, we may want to fit a quadratic predictor. Introducing the coefficient vector

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ w_1 \\ w_2 \end{pmatrix} \in \mathbb{R}^{3 \times 1},$$

such a predictor has the form

$$f_{\theta}^{\text{quad}}(x) := b + w_1x + w_2x^2. \quad (46)$$

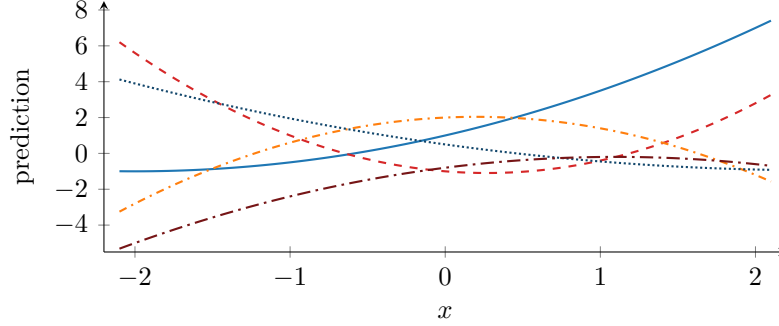


Figure 8: Different choices of $\theta = (b, w_1, w_2)^\top$ give different quadratic predictors f_{θ}^{quad} . These curves show possible members of this family.

At first sight, this may look like a different regression problem. However, the key observation is that the predictor is still linear in the adjustable coefficients b, w_1, w_2 . We can regard the quantities $1, x$, and x^2 as three features of the input. Thus, instead of using a design matrix whose columns contain 1 and x , we simply add a third column containing x^2 . The predictor may now be nonlinear as a function of x , but the fitting problem retains the same structure as before.

This observation motivates a more general construction. Choose fixed functions

$$\phi_1, \dots, \phi_m : \mathbb{R} \rightarrow \mathbb{R},$$

called *features*, and collect them in the *feature map*

$$\phi : \mathbb{R} \longrightarrow \mathbb{R}^{1 \times m}, \quad \phi(x) = \begin{pmatrix} \phi_1(x) & \cdots & \phi_m(x) \end{pmatrix}. \quad (47)$$

Thus, instead of using the original input x directly, we represent it through the m quantities $\phi_1(x), \dots, \phi_m(x)$.

Given a column of coefficients

$$\theta = \begin{pmatrix} \theta_1 \\ \vdots \\ \theta_m \end{pmatrix} \in \mathbb{R}^{m \times 1},$$

we define the predictor

$$f_{\theta}^{\phi}(x) := \phi(x)\theta = \sum_{j=1}^m \theta_j \phi_j(x). \quad (48)$$

Again, the features may be nonlinear functions of the original input, but the predictor remains linear in the adjustable coefficients $\theta_1, \dots, \theta_m$. If one of the features is the constant function 1 , its coefficient plays the role of the intercept, so no additional intercept needs to be introduced separately.

For example, the quadratic predictor (46) corresponds to $\phi_1(x) = 1$, $\phi_2(x) = x$, and $\phi_3(x) = x^2$, so the feature map becomes

$$\phi(x) = \begin{pmatrix} \phi_1(x), \phi_2(x), \phi_3(x) \end{pmatrix} = \begin{pmatrix} 1, x, x^2 \end{pmatrix}.$$

For the observed inputs $x_1, \dots, x_N$, we place the corresponding feature rows into the *transformed design matrix*

$$\mathbf{\Phi} := \begin{pmatrix} \phi(x_1) \\ \vdots \\ \phi(x_N) \end{pmatrix} = \begin{pmatrix} \phi_1(x_1) & \cdots & \phi_m(x_1) \\ \vdots & & \vdots \\ \phi_1(x_N) & \cdots & \phi_m(x_N) \end{pmatrix} \in \mathbb{R}^{N \times m}. \quad (49)$$

Its entries are

$$\Phi_{ij} = \phi_j(x_i).$$

The vector of predictions on the observed inputs is therefore

$$\begin{pmatrix} f_{\boldsymbol{\theta}}^{\phi}(x_1) \\ \vdots \\ f_{\boldsymbol{\theta}}^{\phi}(x_N) \end{pmatrix} = \mathbf{\Phi} \boldsymbol{\theta} \in \mathbb{R}^{N \times 1},$$

and, if the observed outputs are collected in the column vector

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{N \times 1},$$

the corresponding empirical squared loss becomes

$$\mathcal{L}_{\text{sq}}^{\phi}(\boldsymbol{\theta}) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}^{\phi}(x_i), y_i) = \|\mathbf{\Phi} \boldsymbol{\theta} - \mathbf{y}\|_2^2. \quad (50)$$

We have now arrived at exactly the same least-squares problem as in Section 4.2, with $\mathbf{\Phi}$ in place of $\mathbf{X}$. Therefore, all the calculations from that section apply directly. In particular, the stationary points satisfy the normal equations

$$\mathbf{\Phi}^T \mathbf{\Phi} \boldsymbol{\theta} = \mathbf{\Phi}^T \mathbf{y}. \quad (51)$$

As we proved in Section 4.2, every solution of these equations is a global minimiser. If $\mathbf{\Phi}^T \mathbf{\Phi}$ is invertible, the solution is unique and is given by

$$\boldsymbol{\theta}^* = (\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T \mathbf{y}. \quad (52)$$

For the quadratic predictor, the transformed design matrix is simply

$$\mathbf{\Phi} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 \end{pmatrix}. \quad (53)$$

We remark once again that, although the features may be nonlinear functions of the input, this is still a *linear regression* model because the predictor is linear in the coefficients that we fit. For example, (46) is quadratic, and hence nonlinear, as a function of x when $w_2 \neq 0$, but it has the form

$$b \cdot 1 + w_1 \cdot x + w_2 \cdot x^2,$$

which is linear in the adjustable coefficients b, w_1, w_2 .

By contrast, a model such as

$$f_w(x) = \sin(wx)$$

is not linear in the adjustable coefficient w . In particular, it cannot be written in the fixed-feature form

$$f_w(x) = w\phi(x)$$

for a feature ϕ that does not depend on w .²

Worked example. For the same five observations considered in the straight-line example, the quadratic feature map gives

$$\Phi = \begin{pmatrix} 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}.$$

The corresponding normal equations (51) are

$$\underbrace{\begin{pmatrix} 5 & 0 & 10 \\ 0 & 10 & 0 \\ 10 & 0 & 34 \end{pmatrix}}_{\Phi^T \Phi} \boldsymbol{\theta} = \underbrace{\begin{pmatrix} 10 \\ 20 \\ 27 \end{pmatrix}}_{\Phi^T \mathbf{y}}.$$

We can check that the matrix $\Phi^T \Phi$ is invertible, so the solution is unique. Although we could use (52), here it is simpler to solve the normal equations directly by *Gaussian elimination*. In (54), the last column contains the right-hand side. Subtracting twice the first row from the third row gives

$$\left[\begin{array}{ccc|c} 5 & 0 & 10 & 10 \\ 0 & 10 & 0 & 20 \\ 10 & 0 & 34 & 27 \end{array} \right] \xrightarrow{R_3 \leftarrow R_3 - 2R_1} \left[\begin{array}{ccc|c} 5 & 0 & 10 & 10 \\ 0 & 10 & 0 & 20 \\ 0 & 0 & 14 & 7 \end{array} \right]. \quad (54)$$

The coefficient matrix on the right of (54) is upper triangular. Reading its equations from the bottom up, a procedure called *back substitution*, gives

$$\theta_3 = \frac{7}{14} = \frac{1}{2}, \quad \theta_2 = \frac{20}{10} = 2, \quad \theta_1 = \frac{10 - 10\theta_3}{5} = 1. \quad (55)$$

Therefore,

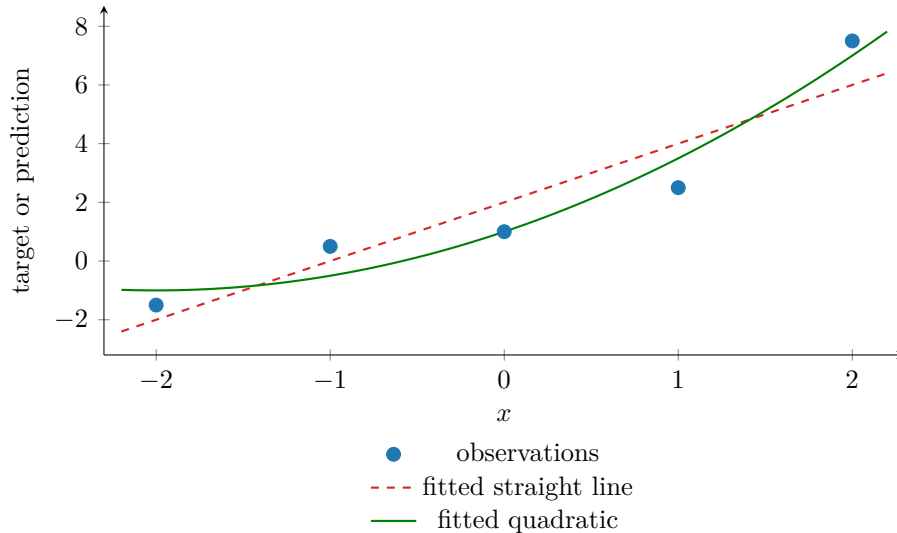
$$\boldsymbol{\theta}^* = \begin{pmatrix} 1 \\ 2 \\ 1/2 \end{pmatrix}, \quad f_{\boldsymbol{\theta}^*}^{\text{quad}}(x) = 1 + 2x + \frac{1}{2}x^2.$$

Its predictions are $-1, -0.5, 1, 3.5, 7$, with residuals $0.5, -1, 0, 1, -0.5$. The squared loss is

$$0.5^2 + (-1)^2 + 0^2 + 1^2 + (-0.5)^2 = \frac{5}{2},$$

which is smaller than the fitted straight-line loss 6.

²Indeed, if there were a function ϕ , independent of w , such that $\sin(wx) = w\phi(x)$ for all w, x , then setting $w = 1$ would give $\phi(x) = \sin x$. Hence we would have $\sin(wx) = w \sin x$ for all w, x , which is false. For example, with $x = \pi/2$ and $w = 2$, the two sides are 0 and 2.



5 Gradient descent

We have seen that the squared-loss minimiser for linear regression can be computed by solving the normal equations. For a dense design matrix with N observations and p fitted coefficients, forming the normal equations costs $O(Np^2)$ arithmetic operations, while solving the resulting $p \times p$ linear system by Gaussian elimination costs $O(p^3)$.³ Here p counts all the fitted coefficients, so $p = d + 1$ for the affine model with d input features.

When the number of coefficients is large, solving the normal equations directly can therefore become expensive. An alternative is to approach the minimiser gradually rather than compute it in one step. This leads us to *gradient descent*, an iterative method for minimising a differentiable objective.

Gradient descent starts from an initial parameter vector and repeatedly uses derivatives to decide how the parameters should be changed in order to reduce the loss. Each update produces a new candidate solution. For linear regression, each such iteration costs $O(Np)$, so reaching a sufficiently good approximation in relatively few iterations may be cheaper than solving the normal equations directly.

Linear regression is also a convenient setting in which to introduce gradient descent because we already know how to compute its minimiser exactly. We can therefore compare the iterates produced by the method with the solution obtained from the normal equations. In later lectures, however, we will use the same idea for models for which no direct solution is available.

We will first introduce the general gradient-descent update and explain why moving against the gradient tends to reduce the objective. We will then apply it to the squared-loss objective for linear regression.

³Here, $O(\cdot)$ indicates an upper bound on how the number of arithmetic operations grows with the problem dimensions, up to a constant factor.

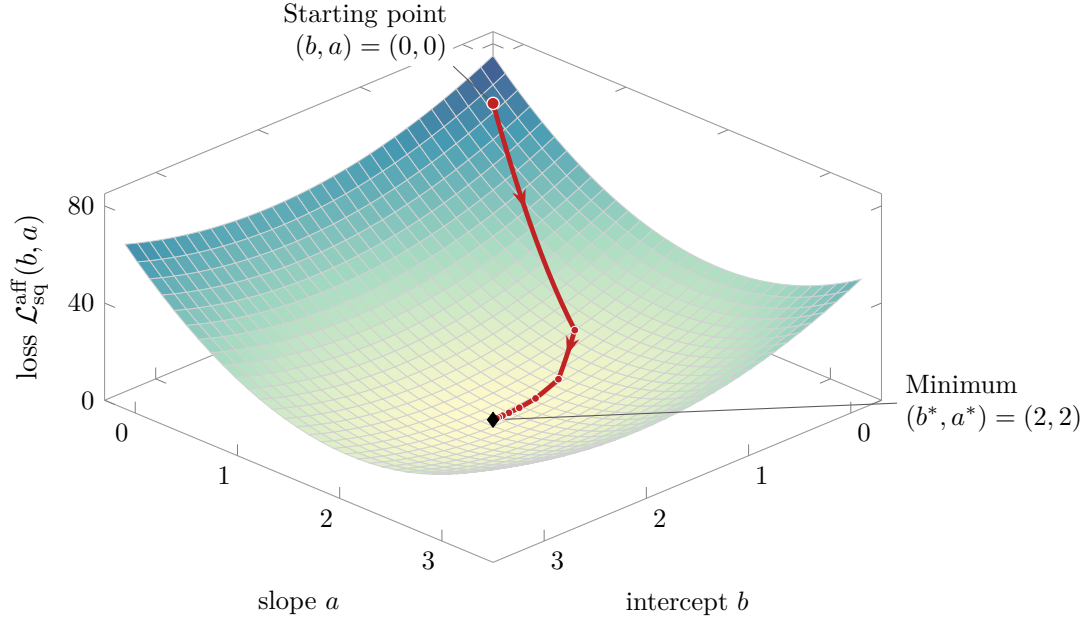


Figure 9: Gradient descent moves downhill on the loss surface. Height represents the squared-error loss for our five observations. The red points are successive iterates, starting from $(b, a) = (0, 0)$ with learning rate $\eta = 0.04$, and the arrows indicate their order. They approach the minimum at $(2, 2)$, where the loss is 6. Eight updates are shown.

Let $\mathcal{L} : \mathbb{R}^p \rightarrow \mathbb{R}$ be a differentiable objective of a column parameter $\boldsymbol{\theta}$. As in Lecture 1, its gradient has the same shape as that parameter: it is the column of its coordinate derivatives. Starting from an initial parameter vector $\boldsymbol{\theta}^{(0)}$, gradient descent updates

$$\boxed{\boldsymbol{\theta}^{(t+1)} := \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})}, \quad t = 0, 1, 2, \dots, \quad (56)$$

where $\eta > 0$ is the *learning rate*, or step size. The superscript (t) is an iteration index, not a power and not an observation index.

5.1 Why move against the gradient?

For a small change $\boldsymbol{\Delta}$, differentiability gives the first-order approximation

$$\mathcal{L}(\boldsymbol{\theta} + \boldsymbol{\Delta}) \approx \mathcal{L}(\boldsymbol{\theta}) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top \boldsymbol{\Delta}. \quad (57)$$

The last term in (57) is the sum of each coordinate change multiplied by the corresponding partial derivative. With $\boldsymbol{\Delta} = -\eta \nabla \mathcal{L}(\boldsymbol{\theta})$, it is

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top \boldsymbol{\Delta} = -\eta \|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2^2.$$

If the gradient is nonzero, this is negative, and therefore a sufficiently small step in that direction reduces the objective. The phrase *sufficiently small* matters: a first-order approximation is only local and does not promise anything about arbitrarily large steps.

The step size matters. An excessively large step can increase the loss. For example, take

$$F(a) = 100a^2, \quad F'(a) = 200a. \quad (58)$$

Applying gradient descent to (58) with $\eta = 0.1$ gives

$$a^{(t+1)} = a^{(t)} - 0.1 \cdot 200a^{(t)} = -19a^{(t)}. \quad (59)$$

Under (59), the magnitude grows from any nonzero starting value. Even with a suitable step size, approaching a minimiser need not mean reaching it in finitely many steps.

5.2 Gradient descent for a straight line

We now apply the gradient-descent rule (56) to the affine predictor

$$f_{\boldsymbol{\theta}}^{\text{aff}}(x) = b + ax, \quad \boldsymbol{\theta} = \begin{pmatrix} b \\ a \end{pmatrix}.$$

The objective to minimise is the empirical squared loss $\mathcal{L}_{\text{sq}}^{\text{aff}}$ defined in (20). Thus, in the general update (56), we replace $\mathcal{L}$ by $\mathcal{L}_{\text{sq}}^{\text{aff}}$:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}^{(t)}). \quad (60)$$

To perform this update, we need to calculate the gradient at the current parameter column

$$\boldsymbol{\theta}^{(t)} = \begin{pmatrix} b^{(t)} \\ a^{(t)} \end{pmatrix}.$$

Since the first coordinate is the intercept b and the second is the slope a , the gradient contains the partial derivative with respect to b first, followed by the partial derivative with respect to a .

We have already calculated these derivatives in (22) and (23). Previously, we set them equal to zero to find the stationary point. This time, we evaluate them at the current values $b^{(t)}, a^{(t)}$ to determine the update.

For each observation, first calculate the residual of the current line:

$$r_i^{(t)} := b^{(t)} + a^{(t)}x_i - y_i, \quad i = 1, \dots, N. \quad (61)$$

Evaluating (22) and (23) at $(b^{(t)}, a^{(t)})$, and then using (61), gives

$$\nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}^{(t)}) = \begin{pmatrix} \frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial b}(b^{(t)}, a^{(t)}) \\ \frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial a}(b^{(t)}, a^{(t)}) \end{pmatrix} = 2 \begin{pmatrix} \sum_{i=1}^N (b^{(t)} + a^{(t)}x_i - y_i) \\ \sum_{i=1}^N x_i (b^{(t)} + a^{(t)}x_i - y_i) \end{pmatrix} = 2 \begin{pmatrix} \sum_{i=1}^N r_i^{(t)} \\ \sum_{i=1}^N x_i r_i^{(t)} \end{pmatrix}. \quad (62)$$

We can now substitute (62) into (60). The resulting update is

$$\begin{pmatrix} b^{(t+1)} \\ a^{(t+1)} \end{pmatrix} = \begin{pmatrix} b^{(t)} \\ a^{(t)} \end{pmatrix} - 2\eta \begin{pmatrix} \sum_{i=1}^N r_i^{(t)} \\ \sum_{i=1}^N x_i r_i^{(t)} \end{pmatrix}. \quad (63)$$

Hence, at each iteration, first compute the residuals in (61) and then write (63) component by component:

$$\begin{aligned} r_i^{(t)} &:= b^{(t)} + a^{(t)}x_i - y_i, & i = 1, \dots, N, \\ b^{(t+1)} &:= b^{(t)} - 2\eta \sum_{i=1}^N r_i^{(t)}, \end{aligned} \tag{64}$$

$$a^{(t+1)} := a^{(t)} - 2\eta \sum_{i=1}^N x_i r_i^{(t)}. \tag{65}$$

Both updates use the same residuals, computed before either coefficient is changed. We only recompute them after updating both coefficients.

Interpreting the signs. Recall that the residual at iteration t is

$$r_i^{(t)} = f_{\theta^{(t)}}(x_i) - y_i.$$

Therefore, $r_i^{(t)} < 0$ means that the current prediction is below the target y_i , while $r_i^{(t)} > 0$ means that it is above the target.

Consider first the update of the intercept. If

$$\sum_i r_i^{(t)} < 0,$$

then the predictions lie below their targets on average. Since gradient descent subtracts this sum in update (64), the new intercept $b^{(t+1)}$ is larger than $b^{(t)}$. If the slope were kept fixed, this increase in the intercept would shift every prediction upwards. In the full gradient-descent update, however, the slope changes as well, so the total change in a prediction depends on both coefficient updates.

To interpret the slope update, it is useful to consider the contribution of one observation at a time. The actual change from $a^{(t)}$ to $a^{(t+1)}$, however, depends on the sum of these contributions over all observations.

For the slope, the sign of the input also matters. Suppose first that $x_i > 0$ and that the current prediction at observation i is below its target, so $r_i^{(t)} < 0$. Then

$$x_i r_i^{(t)} < 0.$$

This observation therefore gives a negative contribution to the gradient with respect to the slope. Since gradient descent subtracts the gradient, this contribution pushes the update towards a larger value of $a^{(t+1)}$. At a positive value of x_i , increasing the slope tends to increase the prediction at that observation, moving it towards the target.

If instead $x_i < 0$ while the current prediction is still below its target, then

$$x_i r_i^{(t)} > 0.$$

The same observation now gives a positive contribution to the gradient with respect to the slope. Since gradient descent subtracts the gradient, this contribution pushes the update towards a smaller value of $a^{(t+1)}$. At a negative value of x_i , decreasing the slope tends to increase the prediction at that observation, again moving it towards the target.

Analogous reasoning applies when $r_i^{(t)} > 0$, with the direction of the slope update again depending on the sign of x_i .

Worked example: one simultaneous update. Use our five observations, start from

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} b^{(0)} \\ a^{(0)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

and take $\eta = 0.04$, as in Figure 9. The initial line predicts zero at every input. Therefore, by (61), its residuals are

$$r_1^{(0)} = 1.5, \quad r_2^{(0)} = -0.5, \quad r_3^{(0)} = -1, \quad r_4^{(0)} = -2.5, \quad r_5^{(0)} = -7.5.$$

The two sums needed in (62) are

$$\sum_{i=1}^5 r_i^{(0)} = -10, \quad \sum_{i=1}^5 x_i r_i^{(0)} = -20.$$

Hence,

$$\nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}^{(0)}) = 2 \begin{pmatrix} -10 \\ -20 \end{pmatrix} = \begin{pmatrix} -20 \\ -40 \end{pmatrix}. \quad (66)$$

Substituting (66) into (60) gives

$$\boldsymbol{\theta}^{(1)} = \begin{pmatrix} b^{(1)} \\ a^{(1)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 0.04 \begin{pmatrix} -20 \\ -40 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 1.6 \end{pmatrix}. \quad (67)$$

The new line is therefore

$$f_{\boldsymbol{\theta}^{(1)}}^{\text{aff}}(x) = 0.8 + 1.6x.$$

Its predictions are $-2.4, -0.8, 0.8, 2.4, 4$, with residuals $-0.9, -1.3, -0.2, -0.1, -3.5$. The empirical loss decreases from 66 to

$$(-0.9)^2 + (-1.3)^2 + (-0.2)^2 + (-0.1)^2 + (-3.5)^2 = 14.8.$$

The parameters are not optimal after one step: the analytic solution was $(b^*, a^*) = (2, 2)$, with empirical loss 6. To perform the next iteration, we repeat the same calculation using the residuals of the new line.

5.3 Gradient descent with several features

The straight-line update extends directly to the model from Section 4. Its coefficient column $\boldsymbol{\theta} = (b, \mathbf{w}^\top)^\top$ already contains the intercept, and the first column of $\mathbf{X}$ already contains the ones that multiply it. We can therefore update all the coefficients using the single gradient formula (41).

At iteration t , multiplying $\mathbf{X}$ by the current coefficient column $\boldsymbol{\theta}^{(t)}$ gives the current predictions. Subtracting the targets gives the corresponding residuals. Therefore, using formula (41), the gradient at the current coefficients is

$$\nabla \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}^{(t)}) = 2\mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta}^{(t)} - \mathbf{y}). \quad (68)$$

Previously, we set this gradient equal to zero in order to solve the fitting problem directly. Now, instead, we use its value at the current coefficients to decide how those coefficients should change. Substituting (68) into the gradient-descent rule (56) gives

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - 2\eta \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta}^{(t)} - \mathbf{y})}. \quad (69)$$

Recognising that $\mathbf{X}\boldsymbol{\theta}^{(t)} - \mathbf{y}$ is precisely the residual column at iteration t , we can equivalently write the update as two successive steps. Specifically, first compute the residuals using the current coefficients, and then use those residuals to update all the coefficients:

$$\begin{array}{ll} \text{Residual:} & \mathbf{r}^{(t)} := \mathbf{X}\boldsymbol{\theta}^{(t)} - \mathbf{y}, \\ \text{Parameters:} & \boldsymbol{\theta}^{(t+1)} := \boldsymbol{\theta}^{(t)} - 2\eta\mathbf{X}^\top\mathbf{r}^{(t)}. \end{array} \quad (70)$$

This two-step form makes the computation explicit: all the residuals are first evaluated using $\boldsymbol{\theta}^{(t)}$, and the resulting column is then used to update the whole parameter vector at once.

The residual column has shape $N \times 1$, while $\mathbf{X}^\top\mathbf{r}^{(t)}$ has shape $(d+1) \times 1$. Thus the update changes one coefficient per entry, including the intercept in the first entry. All these changes use the same residuals, computed before any coefficient is updated. Only afterwards do we recompute the predictions and residuals for the next iteration.

For a straight line, the columns of $\mathbf{X}$ are the ones and the input values, so the product in (70) is

$$\mathbf{X}^\top\mathbf{r}^{(t)} = \begin{pmatrix} \sum_{i=1}^N r_i^{(t)} \\ \sum_{i=1}^N x_i r_i^{(t)} \end{pmatrix}.$$

We therefore recover both updates from Section 5.2 at once.

Each iteration needs two matrix–vector products: one for the predictions and one for the gradient. With N observations and $d+1$ coefficients, their total cost is $O(N(d+1))$ arithmetic operations.

5.4 Gradient descent with nonlinear features

The same reasoning from Section 5.3 applies to the fixed nonlinear features introduced in Section 4.5. There, the transformed design matrix is Φ , and the vector of predictions is $\Phi\boldsymbol{\theta}$. Since the squared-loss objective in (50) has the same form as the ordinary least-squares objective, we can use the same gradient formula as in (41), with Φ in place of $\mathbf{X}$.

At iteration t , the gradient evaluated at the current coefficients is therefore

$$\nabla\mathcal{L}_{\text{sq}}^\phi(\boldsymbol{\theta}^{(t)}) = 2\Phi^\top(\Phi\boldsymbol{\theta}^{(t)} - \mathbf{y}). \quad (71)$$

As before, we do not set this gradient equal to zero. Instead, we use its value at the current coefficients to decide how those coefficients should change. Substituting (71) into the gradient-descent rule (56) gives

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - 2\eta\Phi^\top(\Phi\boldsymbol{\theta}^{(t)} - \mathbf{y}). \quad (72)$$

Recognising that $\Phi\boldsymbol{\theta}^{(t)} - \mathbf{y}$ is the residual column at iteration t , we can equivalently write the update as two successive steps: first compute the residuals using the current coefficients, and then use those residuals to update all the coefficients:

$$\begin{array}{ll} \text{Residual:} & \mathbf{r}^{(t)} := \Phi\boldsymbol{\theta}^{(t)} - \mathbf{y}, \\ \text{Parameters:} & \boldsymbol{\theta}^{(t+1)} := \boldsymbol{\theta}^{(t)} - 2\eta\Phi^\top\mathbf{r}^{(t)}. \end{array} \quad (73)$$

This is exactly the same gradient-descent procedure as in (70), with the transformed design matrix Φ in place of the original design matrix $\mathbf{X}$.

For the quadratic features in (53), the three columns of Φ contain 1, x , and x^2 . The first coefficient therefore plays the role of the intercept, while the other two multiply x and x^2 . Thus, choosing nonlinear features changes the family of predictors that we can fit, but it does not require a different gradient-descent procedure.

6 Exercises

The first three exercises and Exercise 6 use the dataset below. The exercises can be done by hand with a calculator. Use the sum of squared errors, without a factor of $1/2$ or $1/N$, unless another objective is explicitly stated.

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-6.2	-2.6	0.5	2.7	5.7

1. Compare three predictors.

Compute the predictions and sum-of-squared-errors value for each function

$$f_1(x) = x, \quad f_2(x) = 2x, \quad f_3(x) = 3x.$$

Which gives the smallest loss on the displayed dataset? Does this establish that it is the best predictor among all straight lines?

2. Fit a straight line.

Collect the intercept and slope in the column vector $\theta = (b, a)^\top$, and fit the family

$$f_\theta^{\text{aff}}(x) = b + ax$$

by squared-error minimisation, using the matrix method in Section 4. Construct $\mathbf{X}$ with a first column of ones and a second column containing the inputs. Then compute $\mathbf{X}^\top \mathbf{X}$ and $\mathbf{X}^\top \mathbf{y}$, and solve the normal equations for θ . Check that the minimiser is unique.

3. Fit a quadratic without an intercept.

Collect the two coefficients in the column vector $\theta = (w_1, w_2)^\top$, and fit the predictor

$$f_\theta(x) := w_1 x + w_2 x^2$$

by the same squared-error criterion. Specify the two feature functions, form Φ , and compute the fitted coefficients. This family has no constant term, so do not add a column of ones. Check that $\Phi^\top \Phi$ is invertible, as required to apply formula (52).

4. Differentiate the absolute value.

Let $q : \mathbb{R} \rightarrow \mathbb{R}$ be $q(s) = |s|$. Sketch its graph and find $q'(s)$ separately for $s < 0$ and $s > 0$. Explain why the derivative does not exist at zero. Express the derivative at nonzero inputs using

$$\text{sign}(s) = \begin{cases} -1, & s < 0, \\ 0, & s = 0, \\ 1, & s > 0. \end{cases} \quad (74)$$

The convention $\text{sign}(0) = 0$ in (74) is not a statement that $q'(0)$ exists.

5. Changing the loss for the constant predictor.

Return to the five observations used in the worked examples:

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-1.5	0.5	1	2.5	7.5

In Section 3.1, we fitted the constant predictor from (7) using the squared loss ℓ_{sq} from (4). We found that the best constant is the mean $\bar{y} = 2$, with $\mathcal{L}_{\text{sq}}^{\text{const}}(2) = 46$.

What changes if we use the absolute loss ℓ_{abs} from (4) instead? Keeping the same data and family of predictors, we now minimise

$$\mathcal{L}_{\text{abs}}^{\text{const}}(b) = \sum_{i=1}^5 \ell_{\text{abs}}(f_b^{\text{const}}(x_i), y_i) = \sum_{i=1}^5 |b - y_i|. \quad (75)$$

- For b different from all five targets, express the derivative of (75) using the sign function (74) from Exercise 4. Interpret it as the number of targets below b minus the number above it. Find the slope on each interval between successive targets, and on the two outer intervals. Sketch $\mathcal{L}_{\text{abs}}^{\text{const}}$, find its minimiser b_{abs}^* , and relate it to the median of the five targets. Does $(\mathcal{L}_{\text{abs}}^{\text{const}})'(b_{\text{abs}}^*)$ exist?
- Compare your fitted constant with the squared-loss fit already obtained in Section 3.1. Complete the table, where the known squared-error value is supplied:

constant	$\mathcal{L}_{\text{abs}}^{\text{const}}(b)$	$\mathcal{L}_{\text{sq}}^{\text{const}}(b)$
b_{abs}^*		
$\bar{y} = 2$		46

Which constant is preferred under each loss? Compare entries within each column, because the two columns measure error differently.

- Plot the five observations and the two horizontal prediction lines $y = b_{\text{abs}}^*$ and $y = \bar{y}$ on the same axes. Explain how changing the loss has changed the optimal predictor, referring to the way the two losses penalise large errors in Section 2.

Reflect on what this comparison shows: choosing a loss determines what we optimise for and can change which predictor is best. If the loss does not reflect the goal of the application, even a predictor that minimises it may be a poor choice for that goal.

Hint for (a). As b crosses one target from left to right, that term's slope changes from -1 to $+1$. At a corner, inspect the slopes on either side, rather than assuming that a minimiser must have a derivative.

6. One gradient-descent update.

Return to the dataset at the beginning of this section and the affine predictor from Exercise 2. Start from

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} b^{(0)} \\ a^{(0)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

and take the learning rate $\eta = 0.04$.

Compute the initial residuals and use (64) and (65) to obtain $\boldsymbol{\theta}^{(1)}$. Both coefficient updates must use the same initial residuals. Check your result using the matrix update (70).

Compute the empirical squared loss before and after the update. Has it decreased? Compare the updated coefficients and loss with the minimiser found in Exercise 2.

A Invertibility of the normal-equation matrix

Let $\mathbf{X} \in \mathbb{R}^{N \times m}$. For the affine design matrix in Section 4, $m = d + 1$, because the first column contains the ones for the intercept. We use m here so that the same argument also covers a different choice of fixed features.

We will show that

$$\mathbf{X}^\top \mathbf{X} \text{ is invertible} \iff \text{the columns of } \mathbf{X} \text{ are linearly independent.}$$

Recall that the columns of $\mathbf{X}$ are linearly independent if the only linear combination of them that gives the zero vector is the one in which all coefficients are zero. Equivalently,

$$\mathbf{X}\mathbf{v} = \mathbf{0} \implies \mathbf{v} = \mathbf{0}.$$

We first suppose that the columns of $\mathbf{X}$ are linearly independent. We use the following standard linear-algebra result: a square matrix $\mathbf{G}$ is invertible if and only if the only solution of

$$\mathbf{G}\mathbf{v} = \mathbf{0}$$

is $\mathbf{v} = \mathbf{0}$.

Thus, take a column vector $\mathbf{v} \in \mathbb{R}^m$ satisfying

$$\mathbf{X}^\top \mathbf{X}\mathbf{v} = \mathbf{0}. \tag{76}$$

Multiplying (76) on the left by $\mathbf{v}^\top$ gives

$$0 = \mathbf{v}^\top \mathbf{X}^\top \mathbf{X}\mathbf{v} = (\mathbf{X}\mathbf{v})^\top (\mathbf{X}\mathbf{v}) = \|\mathbf{X}\mathbf{v}\|_2^2. \tag{77}$$

The squared norm in (77) is a sum of squared coordinates. It can be zero only when every coordinate is zero. Therefore,

$$\mathbf{X}\mathbf{v} = \mathbf{0}. \tag{78}$$

This is where we use the independence of the columns. Writing them as $\mathbf{c}_1, \dots, \mathbf{c}_m \in \mathbb{R}^N$, equation (78) means

$$v_1 \mathbf{c}_1 + \dots + v_m \mathbf{c}_m = \mathbf{0}. \tag{79}$$

By linear independence, all coefficients in (79) must be zero:

$$v_1 = \dots = v_m = 0.$$

Hence $\mathbf{v} = \mathbf{0}$. We have therefore proved

$$\mathbf{X}^\top \mathbf{X}\mathbf{v} = \mathbf{0} \implies \mathbf{v} = \mathbf{0},$$

which shows that $\mathbf{X}^\top \mathbf{X}$ is invertible.

Conversely, suppose that the columns of $\mathbf{X}$ are not linearly independent. Then there exists a nonzero vector $\mathbf{v} \in \mathbb{R}^m$ such that

$$\mathbf{X}\mathbf{v} = \mathbf{0}.$$

Multiplying by $\mathbf{X}^\top$ gives

$$\mathbf{X}^\top \mathbf{X}\mathbf{v} = \mathbf{X}^\top \mathbf{0} = \mathbf{0}.$$

Thus, $\mathbf{X}^\top \mathbf{X}$ has a nonzero vector in its null space, and therefore it is not invertible.

We conclude that

$\mathbf{X}^\top \mathbf{X} \text{ is invertible} \iff \text{the columns of } \mathbf{X} \text{ are linearly independent.}$
